

Desenvolvedor Back-End [.Net] T1-M1S8

Mini-Projeto Avaliativo - Módulo 01 - Semana 08

SUMÁRIO

1. CONTEXTUALIZAÇÃO	1
2. DESAFIO	2
3. RESULTADOS ESPERADOS (ENTREGA)	2
4. REQUISITOS DAS TAREFAS	3
4.1. Organização da Solução .NET	3
4.2. Requisitos Funcionais (RFs)	3
RF01 - Modelo do Item de Vistoria (ItemVistoria)	3
RF02 - Classe Base Veiculo	3
RF03 - Herança e Checklists Específicos	4
RF04 - Tabela de Pontuação por Item	4
RF05 - Cálculo do Percentual de Aprovação	4
RF06 - Classificação do Estado do Veículo	4
RF07 - Relatório de Pendências e Apontamentos	5
RF08 - Recomendação de Serviços da Oficina	5
RF09 - Varredura de Listas com Laços Tradicionais	5
RF10 - Exibição no Terminal (Console Application)	5
RF11 - Menu Principal e Navegação (Console.ReadLine)	7
4.3. REQUISITOS TÉCNICOS	8
4.5. VERSIONAMENTO COM GITHUB	9
4.6. GRAVAÇÃO DE VÍDEO	9
5. CRITÉRIOS DE AVALIAÇÃO	10
6. CHECKLIST FINAL DE ENTREGA	11
7. MODELO DE README.MD (SUGESTÃO)	12

1. CONTEXTUALIZAÇÃO

No setor automotivo (concessionárias, locadoras de veículos e seguradoras), a **vistoria técnica** é um processo indispensável. Antes de comprar um carro usado para revenda ou aceitá-lo como entrada na troca por um zero-quilômetro, a empresa precisa verificar rigorosamente se o veículo cumpre os requisitos mínimos de segurança e conservação mecânica.

Neste mini-projeto, você (ou sua squad) assumirá o papel de **Desenvolvedor(a) Back-End .NET Júnior** contratado(a) por uma rede de concessionárias. Sua missão é construir uma *engine* (motor de processamento) via **Console Application em C#** que receba a inspeção técnica de um veículo, aplique as regras do **Checklist de Vistoria**, calcule a pontuação final

de aproveitamento, classifique a situação do veículo e emita um relatório detalhado com os serviços recomendados para a oficina.

Este projeto visa consolidar os conteúdos de C# e POO estudados:

- Lógica de programação e tipos primitivos (`string`, `int`, `double`, `bool`);
- Manipulação de coleções (`List<T>`);
- Estruturas de controle (`if/else`, `switch`, `foreach`, `for`);
- Programação Orientada a Objetos: Classes, Atributos, Propriedades, Construtores, Encapsulamento, Uso do `this`, Herança (`:`), sobreescrita (`virtual/override`) e Polimorfismo;
- Versionamento com Git/GitHub

2. DESAFIO

Desenvolver uma aplicação de console em C# chamada:

AutoCheck.ConsoleApp: Motor de Vistoria Veicular

O sistema deverá processar a vistoria de diferentes tipos de veículos (`Carro`, `Moto` e `Caminhao`), onde cada tipo possui seus próprios atributos e seu **Checklist de Inspeção Específico**. Para cada item inspecionado, o técnico atribuirá um status: **"Bom"**, **"Regular"** ou **"Ruim"**.

A aplicação deverá analisar:

- Quais itens foram avaliados como **"Bom"**, **"Regular"** e **"Ruim"**;
- A **pontuação total atingida** em relação à pontuação máxima possível;
- O **percentual de aprovação (%)** do veículo;
- A **classificação final** do veículo (*Aprovado com Excelência*, *Aprovado com Aparentamentos* ou *Reprovado na Vistoria*);
- A **lista de manutenções/revisões** que a oficina precisará realizar.

3. RESULTADOS ESPERADOS (ENTREGA)

Ao final do mini-projeto, você (ou sua squad de até 3 pessoas) deverá entregar:

1. **Repositório Público no GitHub** contendo a solução .NET Core Console. **Histórico de Commits** estruturado com mensagens descritivas no imperativo (ex: *Adiciona classe Moto e checklist especifico*).
2. **Arquivo README.md** explicando o projeto, regras de negócio e instruções para execução via `dotnet run`.
3. **Vídeo de até 5 minutos** apresentando o funcionamento do programa no terminal, demonstrando o código e explicando as decisões de POO.

Entrega no AVA

- A entrega **deverá ser submetida na tarefa do AVA:** "Módulo 01 - Mini-Projeto Avaliativo"
- **Prazo:** 24/08/2026 até às 22h
- **Peso do mini-projeto na nota:** Avaliação M1.1 – 25% da nota do módulo
- **Links obrigatórios**
 - repositório público no GitHub;
 - vídeo demonstrando o funcionamento.

4. REQUISITOS DAS TAREFAS

4.1. Organização da Solução .NET

A estrutura recomendada do repositório é:

```
autocheck-dotnet/  
├── src/  
│   ├── AutoCheck.ConsoleApp/  
│   │   ├── Program.cs  
│   │   ├── Models/  
│   │   │   ├── ItemVistoria.cs  
│   │   │   ├── Veiculo.cs  
│   │   │   ├── Carro.cs  
│   │   │   ├── Moto.cs  
│   │   │   └── Caminhao.cs  
│   │   ├── Services/  
│   │   │   └── MotorVistoria.cs  
│   │   └── AutoCheck.ConsoleApp.csproj  
└── README.md
```

4.2. Requisitos Funcionais (RFs)

RF01 - Modelo do Item de Vistoria (**ItemVistoria**)

Criar uma classe **ItemVistoria** com as propriedades **Nome** (**string**) e **Status** (**string**), aceitando apenas os valores: "Bom", "Regular" ou "Ruim".

RF02 - Classe Base **Veiculo**

Criar a classe abstrata/base **Veiculo** contendo:

- Propriedades: **Marca**, **Modelo**, **Ano**, **Quilometragem** e **List<ItemVistoria> VistoriaRealizada**;
- Construtor explícito utilizando a palavra-chave **this** para atribuição das propriedades;
- Método **AdicionarItemVistoriado(string nome, string status)**;

- Método `virtual List<string> ObterChecklistObrigatorio()` que retorna itens genéricos de inspeção (ex: "Nível de Óleo do Motor", "Bateria e Sistema Elétrico", "Documentação Regularizada").

RF03 - Herança e Checklists Específicos

Criar 3 subclasses que herdam (:) de `Veiculo`:

1. **Carro**: Adiciona a propriedade `QuantidadePortas (int)` e sobrescreve (`override`) o método `ObterChecklistObrigatorio()` adicionando itens de carros (ex: "Estepe e Macaco", "Triângulo de Sinalização", "Ar Condicionado Funcional").
2. **Moto**: Adiciona a propriedade `Cilindradas (int)` e sobrescreve `ObterChecklistObrigatorio()` com itens de moto (ex: "Kit Transmissão/Corrente", "Manetes de Freio/Embreagem", "Pezinho Lateral").
3. **Caminhao**: Adiciona as propriedades `QuantidadeEixos (int)` e `CapacidadeCargaToneladas (double)`, sobrescrevendo `ObterChecklistObrigatorio()` com itens pesados (ex: "Tacógrafo", "Sistema de Freios a Ar", "Trava e Lona da Caçamba").

RF04 - Tabela de Pontuação por Item

No processamento da vistoria, o sistema deve converter o status do item na seguinte pontuação:

- Status **"Bom"**: 10 pontos
- Status **"Regular"**: 5 pontos
- Status **"Ruim"**: 0 pontos

RF05 - Cálculo do Percentual de Aprovação

O sistema deve somar a pontuação atingida e comparar com a pontuação máxima possível (Total de Itens×10), utilizando a fórmula:

Percentual (%) = (PontuaçãoMaximaPossivel / PontuaçãoObtida) × 100

 **Atenção:** Certifique-se de realizar o cast (`double`) na divisão de inteiros em C# para evitar que o valor seja truncado para zero.

RF06 - Classificação do Estado do Veículo

O sistema deve classificar o resultado usando estruturas de decisão (`if/else` ou `switch`):

Percentual Calculado	Classificação Final		Ação Corporativa		
90% a 100%	Aprovado	com	Liberado	para	compra/revenda
	Excelência		imediate.		

60% a 89%	Aprovado com Apontamentos	Exige desconto na compra para reparos da oficina.
0% a 59%	Reprovado na Vistoria	Veículo recusado pela concessionária.

RF07 - Relatório de Pendências e Apontamentos

O sistema deve listar separadamente no terminal:

- **Itens Críticos / Reprovados (Status "Ruim"):** Exigem troca/reparo obrigatório.
- **Itens de Atenção (Status "Regular"):** Exigem revisão preventiva.

RF08 - Recomendação de Serviços da Oficina

Com base nos itens ruins e regulares, o programa deve emitir uma orientação em texto informando os serviços prioritários que a equipe mecânica deverá executar.

RF09 - Varredura de Listas com Laços Tradicionais

Toda a busca, filtragem e cálculo de pontuação deve ser feita utilizando **laços de repetição tradicionais (foreach ou for)** e estruturas decisao (**if**).

RF10 - Exibição no Terminal (Console Application)

O sistema deve imprimir um relatório formatado e limpo no console, exibindo os dados do veículo, dados específicos da subclasse, notas, percentual e parecer final.

Sugestão de output esperado:

```
=====
                        AUTOCHECK.NET - MOTOR DE VISTORIA
=====

[1/2] PROCESSANDO VISTORIA
-----

> DADOS DO VEÍCULO:

- Tipo: Carro

- Modelo: Toyota Corolla 2.0 Flex

- Ano: 2021 | Quilometragem: 45.000 km

- Atributo Específico: 4 Portas
```

> AVALIAÇÃO DOS ITENS INSPECIONADOS (5 ITENS):

[OK] Nível de Óleo do Motor ----- Status: Bom (10 pts)

[OK] Bateria e Sistema Elétrico ----- Status: Bom (10 pts)

[!] Ar Condicionado Funcional ----- Status: Regular (5 pts)

[!] Estepe e Macaco ----- Status: Regular (5 pts)

[X] Triângulo de Sinalização ----- Status: Ruim (0 pts)

> RESUMO DA PONTUAÇÃO:

- Pontuação Atingida: 30 de 50 pontos possíveis

- Percentual de Aprovação: 60,0%

- Classificação Final: [APROVADO COM APONTAMENTOS]

> RELATÓRIO DE MANUTENÇÃO E RECOMENDAÇÕES DA OFICINA:

 ITENS CRÍTICOS / REPROVADOS (AÇÃO IMEDIATA):

- Triângulo de Sinalização: Repor equipamento obrigatório ausente/danificado.

 ITENS DE ATENÇÃO (REVISÃO PREVENTIVA):

- Ar Condicionado Funcional: Realizar higienização e checagem do gás refrigerante.

- Estepe e Macaco: Calibrar pneu reserva e verificar funcionamento do macaco.

[2/2] PROCESSANDO VISTORIA

> DADOS DO VEÍCULO:

- Tipo: Caminhão

- Modelo: Volvo FH 540
- Ano: 2019 | Quilometragem: 280.000 km
- Atributo Específico: 3 Eixos | Cap. Carga: 30,0 Toneladas

> AVALIAÇÃO DOS ITENS INSPECIONADOS (5 ITENS):

- [OK] Nível de Óleo do Motor ----- Status: Bom (10 pts)
- [OK] Bateria e Sistema Elétrico ----- Status: Bom (10 pts)
- [OK] Funcionamento do Tacógrafo ----- Status: Bom (10 pts)
- [OK] Trava e Lona da Caçamba ----- Status: Bom (10 pts)
- [OK] Sistema de Freios a Ar ----- Status: Bom (10 pts)

> RESUMO DA PONTUAÇÃO:

- Pontuação Atingida: 50 de 50 pontos possíveis
- Percentual de Aprovação: 100,0%
- Classificação Final: [APROVADO COM EXCELÊNCIA]

> RELATÓRIO DE MANUTENÇÃO E RECOMENDAÇÕES DA OFICINA:

- Nenhuma pendência mecânica identificada. Veículo liberado para operação!

=====

FIM DO PROCESSAMENTO DE VISTORIAS

=====

RF11 - Menu Principal e Navegação (Console.ReadLine)

O sistema deve ser iniciado com um **Menu Principal** que permaneça em execução dentro de um laço de repetição (**while** ou **do-while**), oferecendo as seguintes opções ao usuário:

- 1 - **Realizar Nova Vistoria:**
 - Pergunta o tipo de veículo (**Carro**, **Moto** ou **Caminhão**).

- Instancia a subclasse correspondente (`new Carro(...)`, `new Moto(...)` ou `new Caminhao(...)`).
 - Coleta os dados cadastrais (Marca, Modelo, Ano, Km e atributos específicos da categoria).
 - Percorre o checklist específico da subclasse solicitando o status ("Bom", "Regular" ou "Ruim") de cada item.
 - Salva o veículo finalizado em uma lista central de vistorias (`List<Veiculo>`).
2. **2 - Exibir Relatório das Vistorias:**
- Verifica se existem vistorias registradas na lista.
 - Se a lista estiver vazia, exibe uma mensagem informativa ("*Nenhuma vistoria realizada até o momento*").
 - Se houver vistorias, percorre a lista e exibe o relatório detalhado de cada veículo avaliado (pontuação, percentual, classificação e pendências).
3. **0 - Sair:**
- Encerra a execução do programa.

4.3. REQUISITOS TÉCNICOS

Conteúdo .NET / C#	Obrigatório ?	Como Demonstrar no Código
Console Application	Sim	Projeto C# .NET Core executável no terminal.
Tipos Primitivos	Sim	Uso de <code>string</code> , <code>int</code> , <code>double</code> e <code>bool</code> .
Listas (<code>List<T></code>)	Sim	Armazenar os itens inspecionados no veículo.
Laços (<code>foreach</code> / <code>for</code>)	Sim	Varredura das listas para cálculo de pontos e busca.
Condicionais (<code>if/else</code>)	Sim	Mapear notas ("Bom"=10, "Regular"=5, "Ruim"=0) e faixas de %.
Classes & Objetos	Sim	Instanciação de <code>Carro</code> , <code>Moto</code> , <code>Caminhao</code> com <code>new</code> .
Construtores	Sim	Construtores com parâmetros explicitamente declarados.
Herança (<code>:</code>)	Sim	Classes filhas herdando de <code>Veiculo</code> .

Polimorfismo (override)	Sim	Sobrescrever <code>ObterChecklistObrigatorio()</code> em cada subclasse.	4.5.
Encapsulamento	Sim	Propriedades com <code>{ get; set; }</code> e métodos apropriados.	
Git / GitHub	Sim	Repositório público com branches e commits.	

VERSIONAMENTO COM GITHUB

O projeto deverá ter um repositório público no GitHub.

Commits mínimos:

Mini-Projeto individual:

- mínimo de 5 commits

Mini-Projeto em squad:

- mínimo de 8 commits

As mensagens de commit devem ser descritivas e escritas no imperativo, deixando claro o que cada passo entregou.

4.6. GRAVAÇÃO DE VÍDEO

Além do desenvolvimento das tarefas você deverá gravar um vídeo (ou seu grupo de até 03 alunos, um vídeo por grupo), com tempo **máximo de 5 minutos**, abordando os seguintes questionamentos:

- Qual o objetivo do sistema? E demonstração de funcionamento.
- O que deve ser realizado para executar o sistema?
- Como você organizou as tarefas antes de começar a desenvolver?
- Você acha que faltou algo no seu código que você poderia melhorar?

Você poderá gravar na vertical ou na horizontal. É importante que apareça seu rosto e esteja em um local com boa iluminação e que seja possível ouvir sua voz bem. Para realizar a entrega do vídeo, coloque em uma pasta do Google Drive em modo leitor para qualquer pessoa com o link (ou subo via YouTube como vídeo não listado), e compartilhe o mesmo na submissão do projeto no AVA. Uma dica interessante é você inserir o vídeo no readme.md do seu projeto no repositório do GitHub. É um requisito que qualquer pessoa consiga visualizar seu vídeo via algum link.

Observação: o vídeo não poderá ser elaborado com auxílio de ferramentas de inteligência artificial para a narração.

5. CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto.

O mesmo possui variação de nota de 0 (zero) a 10 (dez) como nota mínima e máxima, e possui peso de 30% sobre a avaliação do módulo 01 do curso.

Serão desconsiderados e atribuída a nota 0 (zero) os projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas.

Lembre-se: Você está livre para utilizar outras soluções como base, mas não é permitida a cópia. Ou seja, você está livre para consultar materiais, documentação, exemplos e ferramentas de apoio, desde que consiga adaptar a solução ao desafio proposto e explicar o código entregue com segurança e confiança.

Uso do GitHub e Readme.md				
Nº	Critério de Avaliação	0	0,50	1,00
01	Repositório GitHub e README.md	Repositório não enviado ou privado.	Repositório público, mas README incompleto ou sem instruções de <code>dotnet run</code> .	Repositório público e README completo com instruções claras de execução.
02	Histórico de Commits	Apenas 1 commit ou sem histórico.	Histórico com menos commits do que o mínimo exigido ou mensagens vagas.	Mínimo de commits atingido com mensagens descritivas no imperativo.

Apresentação do Projeto				
Nº	Critério de Avaliação	0,00	1,00	2,00
03	Gravação de vídeo	Não gravou o vídeo de apresentação.	Gravou o vídeo de apresentação, mas não apresentou todos os requisitos do item 4.6.	O vídeo apresentou todos os requisitos do item 4.6.

Desenvolvimento do Projeto				
Nº	Critério de Avaliação	0,00	0,50	1,00
04	Modelagem da Classe Base (Veículo)	Não usou POO ou colocou todo o código no <code>Program.cs</code> .	Criou a classe <code>Veiculo</code> , mas sem construtor explícito ou sem encapsulamento.	A aplicação apresentou classe <code>Veiculo</code> bem estruturada com propriedades, construtor e lista de itens.
05	Herança e Subclasses (Carro, Moto, Caminhão)	Não utilizou herança.	Criou subclasses, mas não utilizou a sintaxe de herança (<code>:</code>) ou não	Criou as 3 subclasses herdando de <code>Veiculo</code> e com atributos específicos.

			adicionou atributos próprios.	
06	Checklists Polimórficos (virtual/override)	Não separou checklists por tipo de veículo.	Separou os checklists, mas sem sobrescrever o método herdado (virtual/override)	Sobrescreveu ObterChecklistObrigatorio() conforme o polimorfismo esperado em cada subclasse.
07	Modelagem do Item de Vistoria (ItemVistoria)	Utilizou apenas string solta sem criar a classe de modelo.	Criou a classe ItemVistoria, mas não a integrou na lista da classe Veiculo.	Classe ItemVistoria criada e utilizada em composição na lista do veículo.
08	Lógica de Cálculo e Pontuação	Calculou incorretamente ou não atribuiu notas aos status ("Bom", "Regular", "Ruim").	Calculou a pontuação, mas falhou no cast de double ou errou a pontuação máxima.	Calculou o percentual exato, conforme pontuação atingida vs pontuação máxima.
09	Classificação Condicional do Veículo	Não classificou o resultado em faixas.	Classificou, mas com erros nas estruturas if/else ou faixas inconsistentes.	Classificou corretamente em Excelência, Apontamentos ou Reprovado.

6. CHECKLIST FINAL DE ENTREGA

Antes de enviar no AVA, confira:

ENTREGÁVEIS

ENTREGÁVEIS OBRIGATÓRIOS

- [] Repositório criado como Público no GitHub.
- [] Histórico com o número mínimo de commits descritivos.
- [] Projeto compila e roda perfeitamente via terminal (dotnet run).
- [] Arquivo README.md preenchido com a explicação da solução.
- [] Link do Vídeo de Demonstração de até 5 minutos.

CÓDIGO E POO (C#)

- [] Classe ItemVistoria criada com Nome e Status.
- [] Classe base Veiculo criada com construtor e lista de itens.
- [] Subclasses Carro, Moto e Caminhao criadas com herança (:).
- [] Atributos específicos adicionados nas subclasses (QuantidadePortas, Cilindradas, QuantidadeEixos).
- [] Método ObterChecklistObrigatorio() sobrescrito (virtual/override) em cada subclasse.
- [] Cálculo percentual implementado convertendo "Bom" (10), "Regular" (5) e "Ruim" (0).
- [] Estrutura if/else aplicando as faixas de classificação (Excelência, Apontamentos, Reprovado).
- [] Varredura das listas feita estritamente com foreach ou for (sem LINQ).
- [] Relatório completo impresso no Console.

GIT E GITHUB

- [] Fiz pelo menos 5 commits com mensagens descritivas (8, se for em dupla ou trio)

VÍDEO E ENVIO

- ☐ Gravei o vídeo de até 5 minutos mostrando o sistema rodando e explicando o código
- ☐ Falei no vídeo sobre as branches que criei
- ☐ Declarei no vídeo onde usei IA e o que validei
- ☐ Coloquei o vídeo no Google Drive com permissão de visualização para qualquer pessoa com o link
- ☐ Enviei no AVA os links do repositório, do vídeo e do Kanban até 10/08/2026 às 22h

7. MODELO DE README.MD (SUGESTÃO)

O README é a porta de entrada do seu projeto e vale nota (critério 01). Ele precisa responder, com as suas palavras:

- O que o sistema faz e para que serve;
- Como executá-lo, passo a passo, do zero;
- Qual regra de cálculo da compatibilidade você adotou e por quê;
- Qual critério você usou para priorizar as habilidades na recomendação de estudo;
- Quais conceitos do Módulo 01 você aplicou e onde;
- O que é a arquitetura cliente-servidor e como ela aparece no seu projeto;
- O link do vídeo de apresentação.

A organização das seções é sua.